Sara — A Numba-CUDA Ray Tracer

Matěj Zeman*
Supervisor: Ing. Radek Richtr, Ph.D.
Course: NI-PG1 — Počítačová Grafika 1

May 2026

Abstract

This paper presents a simple ray tracer implemented in Python, utilizing Numba CUDA for GPU acceleration. The renderer supports most of the ray-tracing pipeline including diffuse textures, specular, and transmissive materials, area light integration, and ACES-based colour management with Intel Open Image Denoise pass. A key contribution is the development and comparison of three BVH construction strategies—naive triangle enumeration, median splitting, and Surface Area Heuristic (SAH) with binning—demonstrating up to a **5.4**× reduction in triangle intersection tests on complex scenes. The project also documents practical limitations of using a Python-to-CUDA JIT compiler: absence of native GPU sorting, register pressure anomalies across GPU architectures, and float64 propagation from Python's type system.

1 Engine Structure & Implementation Challenges

1.1 Core Architecture

Sara is built around Numba's CUDA JIT compiler, which translates a subset of Python and NumPy into PTX. The rendering pipeline runs primarily on the GPU, with the CPU serving optional support tasks:

- **GPU** — Primary ray generation and iterative secondary/shadow evaluation; iterative BVH traversal via per-thread local stack (`cuda.local.array`); Möller–Trumbore intersection tests; material shading; HDR accumulation. All ray tracing is executed on GPU with the entire kernel launched from CPU host code.
- **CPU** — Scene loading via `tinyobjloader` C++ library (MIT License, copyright Syoyo Fujita) through pybind11 bindings in `utils/`; BVH construction using SAH binning; post-processing (tonemapping, Intel OIDN denoising, SRGB conversion). These steps are optional: the GPU handles the core rendering pipeline end-to-end, while CPU support is invoked for scene preparation and output formatting.

Scene data uses a *struct-of-arrays* layout in contiguous 1D NumPy arrays: BVH nodes (8 `float32` per node), triangles (9 `float32`: 3 vertices), and materials (14 columns: diffuse/specular/emissive/transmission RGB, roughness, IOR). The kernel uses a $16 \times 16 = 256$-thread block, with each thread rendering one pixel. Samples accumulate iteratively across kernel launches, HDR framebuffer reduced at the end. An ACEScg-to-sRGB colour pipeline handles gamut mapping with six tonemapper options (Section 5.1).

1.2 Implementation Challenges

- **No GPU sorting:** Numba lacks cuSort access. SAH construction requires sorting triangles by projected surface-area cost along three axes, forcing the entire BVH builder onto the CPU. This creates a bottleneck for large scenes (e.g., Dragon with 871k triangles, 6.69 s BVH build time on 16 cores).
- **Iterative traversal:** CUDA device functions cannot sustain arbitrary recursion. BVH traversal uses a per-thread local stack (`STACK_SIZE` = 64) to avoid register exhaustion. Deep trees may require tuning this constant.
- **Register pressure / Ampere vs. Ada anomaly:** The fat render kernel carries ray origin, direction, throughput, bounce counter, material state, traversal stack, and accumulation buffers. On Ampere (RTX 3090), high register pressure reduces occupancy and can degrade performance versus Ada Lovelace (RTX 40-series), which appears to handle the register pressure differently. This highlights the importance of `max_registers` tuning per target GPU.
- **Float64 propagation:** Python `float` maps to C `double` in NumPy, and Numba preserves this through JIT. Intermediate expressions (e.g., `1.0 / n`) promote computations to `float64`, causing slow device-side double-precision operations. Mitigated via explicit `float32()` casts and NumPy aliases throughout `constants.py`.
- **Precission issues and epsilons:** Self-shadowing from floating-point imprecision is corrected by offsetting the ray origin along the surface normal by `RAY_EPSILON`. Three distinct epsilon values are used: `DET_EPSILON` (10^{-9}) for the barycentric determinant, `RAY_EPSILON` (10^{-5}) for t-distance and shadow ray origin offsets, and `BARYCENTRIC_EPSILON` (10^{-7}) for barycentric validity. Tuning is critical: too small leaves artifacts, too large causes missed lights.
- **Numba classes:** Numba's `types.ClassType` Python class support on the GPU—is experimental and can cause performance regressions through hidden memory allocations and type dispatch overhead. The project avoids classes entirely, keeping the code close to C-style procedural code with explicit structs-of-arrays and simple function dispatch.

*Student, CTU FIT Prague, `zemanm40@fit.cvut.cz`

2 Advanced Methods

Beyond basic ray tracing: **Free jitter anti-aliasing** samples each pixel `SAMPLES` times with uniform jitter within the pixel footprint. **Intel OIDN denoising** uses the `RT` filter with a single `color` buffer in `hdr` mode (no albedo/normal buffers), denoising HDR pixel colour directly. The **ACEScg colour pipeline** operates in linear space for all shading, with multiple tonemapping strategies available (Section 5.1).

Numba's compilation model: Numba bridges the gap between Python's dynamic type system and CUDA's static execution model. Rather than accepting `nvcc`-style compiler flags, Numba uses Python decorators (`@cuda.jit`, `device=True`, `fastmath=True`) that are evaluated at JIT compilation time. The compiler infers concrete types from NumPy array dtypes and function signatures, then lowers the typed IR to LLVM IR before generating PTX. A key feature is dead code elimination: Python `if FLAG:` blocks with compile-time constants are pruned entirely from the GPU binary, equivalent to C++ `#ifdef`. However, this model introduces subtle pitfalls. NumPy's automatic promotion rules can elevate `float32` intermediates to `float64` when Python scalars (e.g., `1.0`) appear in expressions, forcing slow device-side double-precision instructions. The project mitigates this by wrapping all literals in `float32()` and defining typed aliases in `constants.py` (e.g., `ZERO`, `ONE`).

Memory hierarchy: Numba's CUDA target mirrors the CUDA C++ memory model through explicit decorators. `cuda.shared.array()` provides block-level shared memory declared inside kernel functions (not at file scope). `cuda.local.array()` gives per-thread private storage, but if register pressure exceeds the hardware limit, the compiler spills local arrays to global memory—a significant performance penalty. This forced the iterative BVH traversal design with a fixed `STACK_SIZE` of 64 entries, deliberately bounded to prevent unbounded register consumption.

3 Acceleration Data Structure

3.1 Methodology

GPU-side BVH traversal is iterative, using a per-thread local stack. For each ray, the algorithm visits nodes in depth-first order, testing both children at internal nodes and all primitives at leaf nodes. Per-pixel metrics (node visits, triangle tests, shadow tests, hit status) are recorded for profiling.

3.2 Construction Strategies

Three strategies were implemented on the CPU (with `@njit` decorator):

1. **Naive** — All triangles in a single leaf node. Tests every triangle per ray; impractical beyond a handful of primitives.
2. **Spatial Median** — Splits at the geometric midpoint along the longest axis. Produces a balanced tree quickly but ignores traversal cost asymmetry.
3. **SAH + Binning** — Minimises the cost function

$$C(s) = 1.0 + \frac{N_{\text{left}} \cdot A_{\text{left}} + N_{\text{right}} \cdot A_{\text{right}}}{A_{\text{parent}}}$$

   where A is child AABB surface area and N is the number of triangles in each child. The constant 1.0 represents the traversal cost of the current node. Binning ($B = 8$ bins per axis) avoids evaluating every split point, trading minor quality loss for significantly faster construction.

The exhaustive SAH variant evaluates every triangle as a split candidate, requiring $O(N^2)$ work per node since it recomputes bounding boxes from scratch for each of the N split positions. Binning reduces this to $O(N + B)$ by mapping triangles into $B = 8$ uniform bins along each axis, then computing prefix and suffix accumulations over the bins to evaluate all $B - 1$ inter-bin splits in $O(B)$ time. This drops total construction complexity from $O(N^2)$ to $O(N \cdot \text{depth})$. The 8-bin approximation provides similar traversal quality: Table 4.1 shows nearly identical traversal metrics between exhaustive and binned SAH (775 vs. 892 mean node visits), confirming negligible quality loss.

After construction, the tree is *flattened* into `bvh_nodes` (8 floats per node) and `tri_ids` arrays, enabling O(1) child access on the GPU.

4 Performance Results

Scene	Config	Build (s)	Node	Tri	Shadow	Traverse
bunny	sah-binning	4.16	892.42	77.26	19.82	29.78
bunny	median-split	5.31	2903.43	413.77	19.83	97.69
bunny	no-binning	508.92	775.11	79.97	19.82	25.70
box-spheres	sah-binning	3.39	454.18	52.85	13.34	15.26
box-spheres	median-split	3.38	1244.35	251.61	13.35	44.90
box-spheres	no-binning	3.84	342.40	51.83	13.35	11.73
dragon	sah-binning	6.69	52.39	7.97	10.68	10.53

Table 4.1: Per-pixel mean metrics (1024^2, 16 SPP).

SAH + binning consistently yields the lowest triangle test counts. On bunny, SAH reduces mean triangle tests from 413.77 (median) to 77.26—a **5.4× improvement**. The no-binning variant (testing every split) is dramatically slower to build (508 s vs. 4 s for bunny) but achieves similar traversal quality, confirming the efficiency of the 8-bin approximation. On the dragon scene (871k triangles), exhaustive SAH would take an estimated 90 min (based on bunny's 455 s for 70k triangles, dragon has 12x more triangles). SAH construction is a one-time CPU cost that pays off in GPU render time: on the dragon scene (871k triangles), 6.69 s of CPU construction replaces hundreds of seconds of naive GPU rendering.

4.1 BVH Construction Metrics

Table 4.2 details the resulting BVH trees after SAH construction. The dragon scene requires 6.69 s of CPU construction and produces a tree of 932,985 nodes for 871,322 triangles, with a deeper tree (depth 32) and higher prims/leaf (mean 1.87) due to its higher triangle count.

4.2 Render Time Logs

Full render pipeline timings on an NVIDIA RTX 3090 (Table 4.3). The GPU tonemapper runs integrated into the

Scene	Nodes	Internal	Leaves	Depth	Prims/leaf
bunny	76,907	38,453	38,454	8.3	1.8
box-spheres	2,183	1,091	1,092	5.7	2.0
dragon	932,985	466,492	466,493	32	1.9

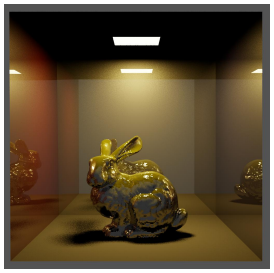
Table 4.2: BVH construction statistics (SAH + binning).

`postprocess_full_gpu_kernel` kernel, combining exposure, ACEScg-to-sRGB 3D LUT lookup, and gamma encoding in a single pass. Without BVH, the bunny scene takes 36 s on GPU (800k tri tests/ray). With SAH BVH, the same scene renders in 0.11 s at 16 SPP with denoising. Shadow rays account for 37–42% of total rays fired, dominating the workload on complex scenes.

Stage	Time (s)
BVH build (bunny)	7.34
Render (bunny, w/ BVH, 16 SPP)	0.09
Tonemap + postprocess (GPU)	0.01
Denoise (GPU, OIDN RT)	0.02
Total render	**0.11**

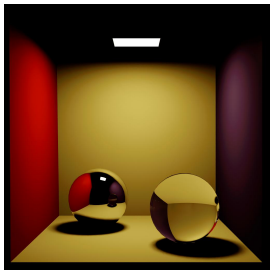
Table 4.3: Timing breakdown (RTX 3090, 1024^2, 16 SPP, denoise ON).

5 Visual Outputs



(a) Bunny

(b) Box-advanced



(c) Box-spheres

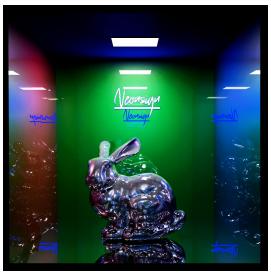
(d) Dragon

Figure 5.1: Scene renders (16 SPP, 1024^2, denoise ON).

5.1 Tonemapper Comparison

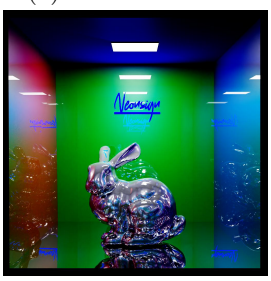
The renderer provides six tonemappers: *custom* (ACEScg → sRGB 3D LUT with Oklab hue protection [6], GPU), *Khronos PBR neutral* (leaves SDR unchanged), *Narkowicz* (per-channel ACES fit, CPU), *Hill* (analytical ACES approximation, CPU), *none* (hard clip), and *magenta* (debug overshoot).

Custom (Oklab hue-preserving 3D LUT [6]). The *custom* path bakes the full ACEScg → sRGB transform



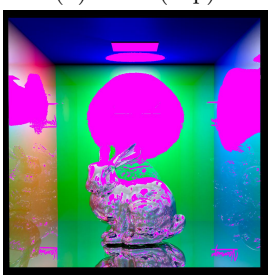
(a) Custom LUT

(b) Khronos neutral



(c) Narkowicz fit

(d) None (clip)



(e) Magenta (debug)

Figure 5.2: Tonemapper comparison on box-advanced scene (16 SPP, denoise ON).

into a pre-generated 32^3 3D LUT, computed at build time by `color-management/generate-lut.py`. The pipeline applies four stages: (i) an ACEScg-to-sRGB chromatic adaptation matrix (3×3 CSC) converts from AP1/D60 to Rec.709/D65 primaries. (ii) A binary-search gamut compression (15 iterations) operates in Oklab space, scaling the chroma channels (a, b) by a per-pixel factor found via bisection on $[0, 1]$ until no negative sRGB channels remain. (iii) The achromatic norm (max RGB channel after gamut mapping) is passed through the Uchimura sigmoidal curve [7], then each channel is scaled by the ratio `mapped_norm / norm`, preserving hue during highlight compression. This decoupling prevents the hue shift that plagues per-channel curves, especially on saturated specular highlights. (iv) A 2.2 linearize decode followed by sRGB gamma encode prepares the result for the gamma LUT lookup. The entire 3D LUT (CSC + gamut compression + Uchimura curve + gamma encode) is evaluated via trilinear interpolation in the `postprocess_full_gpu_kernel` kernel, minimising global memory bandwidth. Gamut insetting was deliberately omitted to preserve maximum vibrancy; rare saturated peaks may still map outside the displayable sRGB gamut.

Khronos PBR neutral. The *Khronos* tonemapper is a per-channel algorithm from the glTF PBR neutral extension. It clamps negatives to zero, applies a black offset adjustment, compresses highlights via a rational function, and desaturates toward white using Hermite interpolation. Because it operates per-channel without gamut compression, saturated HDR values that produce negative linear sRGB during the chromatic adaptation clamp to zero and lose all gradient information, creating flat blobs in intense highlight regions.

Narkowicz / Hill (ACES approximation). Both the *Narkowicz* [3] and *Hill* [4] tonemappers apply the same analytical ACES 1.0 approximation per channel as a rational function:

$$f(x) = \frac{x(2.51x + 0.03)}{x(2.43x + 0.59) + 0.14}$$

The Narkowicz variant [3] provides punchy contrast but can cause slight hue shifts in the brightest regions where channel ratios change during compression. The Hill variant [4] offers slightly smoother highlight roll-off at the cost of reduced mid-tone contrast.

None (hard clip) & Magenta (debug). The *None* mode applies no tonemapping curve, preserving the chromatic adaptation and raw ACEScg saturation of light sources; all shading gradients within values above 1.0 are lost due to hard clipping. The *Magenta* debug mode colors any pixel exceeding 1.0 magenta, serving as a diagnostic for HDR overshoot regions on specular highlights and emissive surfaces.

6 Conclusion

Ray workload analysis on the bunny scene (RTX 3090, BVH ON) shows that primary rays account for 30% of rays, secondary rays 32.8%, and shadow rays 37.2%. The BVH efficiency metric (node/triangle ratio) reaches 14.7:1, yielding a node-to-triangle test ratio of 14.7:1. Per-pixel incidence tests range from 36 to 1379 (mean 115.9), compared to 279,967 without BVH. The *hard* ray fraction (rays with >2× mean tests) comprises 8.6% of the workload, concentrated at specular highlights and refraction boundaries.

Sara demonstrates that a Python-based ray tracer with Numba CUDA achieves high-quality results when combined with struct-of-arrays data layout, SAH + binning BVH construction (reducing triangle tests by over 5× on bunny vs. median splitting), and a complete ACES colour pipeline. Future work includes GPU-side BVH construction (thrust/custom radix sort), volume rendering, and displacement mapping.

Wavefront architecture. An alternative rendering architecture using a grouped execution model (as opposed to the thread-per-pixel model in the main branch) is implemented on the `gpu` branch. This design processes rays in groups of 32, tracking divergence within wavefronts and using thread masks for conditional execution. The wavefront approach can reduce divergence penalties on scenes with high specular/transmission content where rays follow divergent paths. A detailed report with benchmarks is available on the `gpu` git branch.

References

[1] Antti Saavolainen. Ray Tracing Using GPU Computing, 2007. Master's thesis, Eastern Washington University. EWU Digital Commons (PDF)

[2] Syoyo Fujita. TinyObjLoader: Tiny but Powerful Wavefront OBJ Loader, 2016. C++11 single-header library, MIT License. GitHub repository

[3] Krzysztof Narkowicz. ACES Filmic Tone Mapping Curve, Jan 2016. Blog post with analytical ACES approximation and HLSL implementation. Blog post

[4] Stephen Hill. ACES Approximation. ACES fit with a linear denominator, referenced in Narkowicz (2016). RealMJP BakingLab (ToneMapping.hlsl)

[5] Academy of Motion Picture Arts and Sciences and Society of Motion Picture and Television Engineers. Academy Color Encoding Specification, 2021. SMPTE Standard ST 2065-1:2021. IEEE Xplore (SMPTE Standard)

[6] Björn Ottosson. Oklab: A perceptually uniform color space, 2020. Perceptually uniform colour space with luminance-preserving hue protection. Project page

[7] Hajime Uchimura. Tone Mapping Curve Reference Implementation, 2019. Piecewise sigmoidal Uchimura tone mapping curve (GT Tonemapper) with linear toe and exponential highlight roll-off. Desmos Implementation